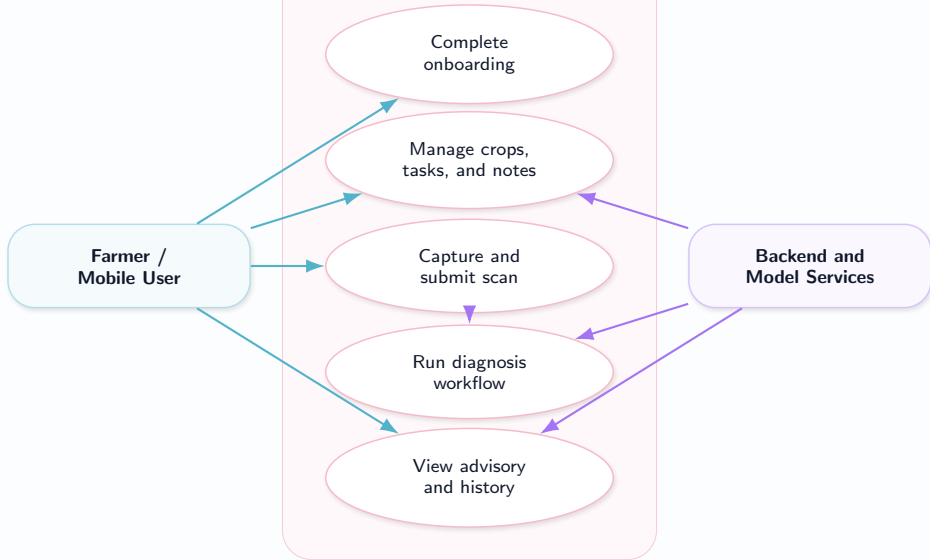
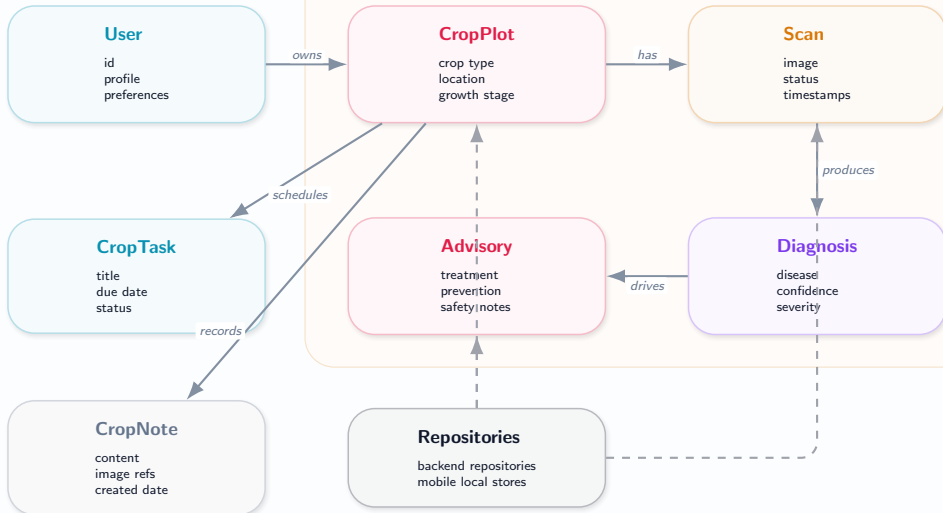


Krishi Vaidya System Boundary



Core Diagnostic Flow Entities



Mobile App

Backend API

VLM Service

Database /
Local Store

submit image and metadata

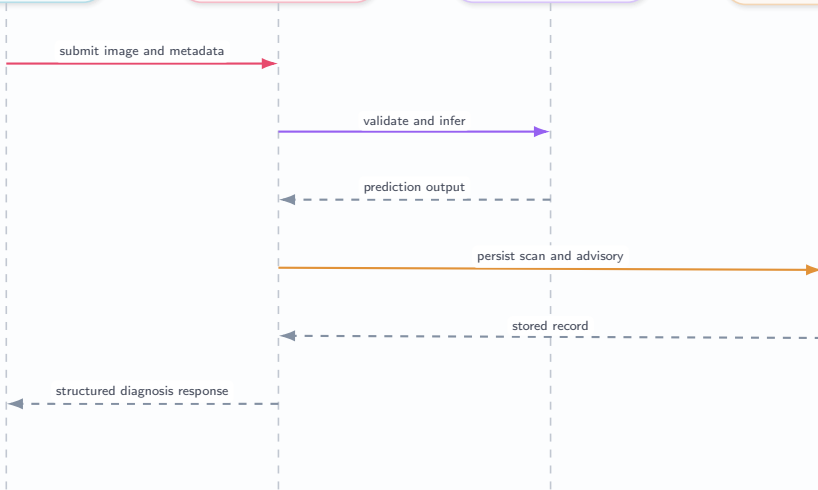
validate and infer

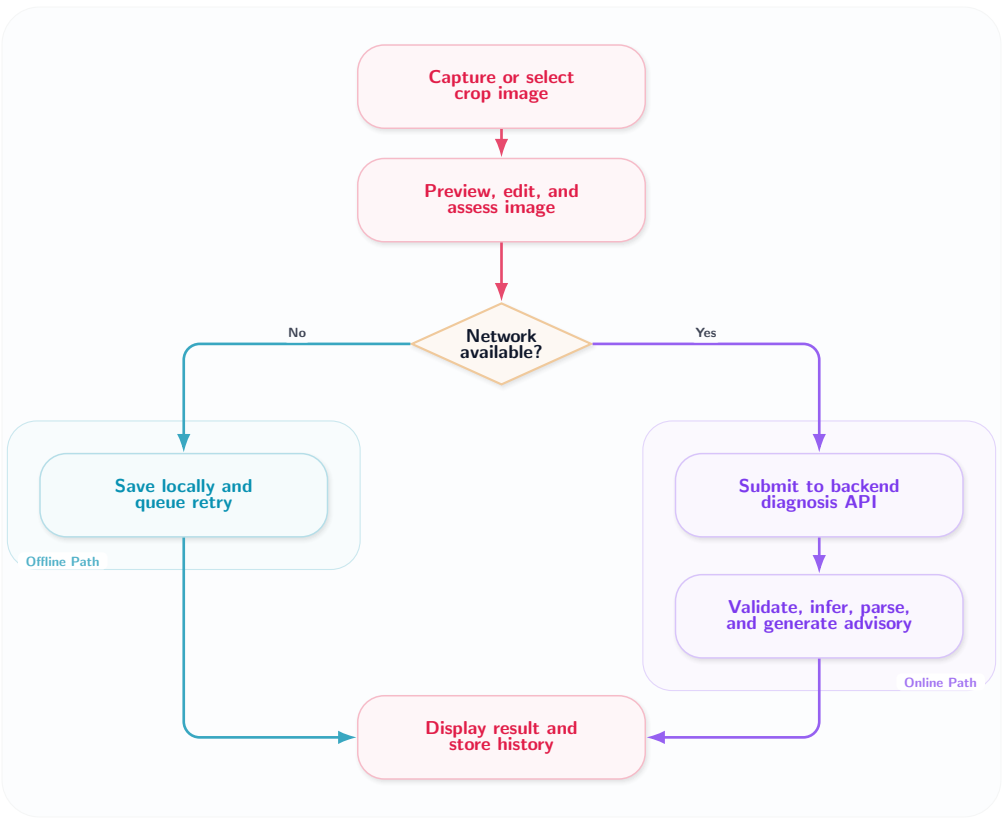
prediction output

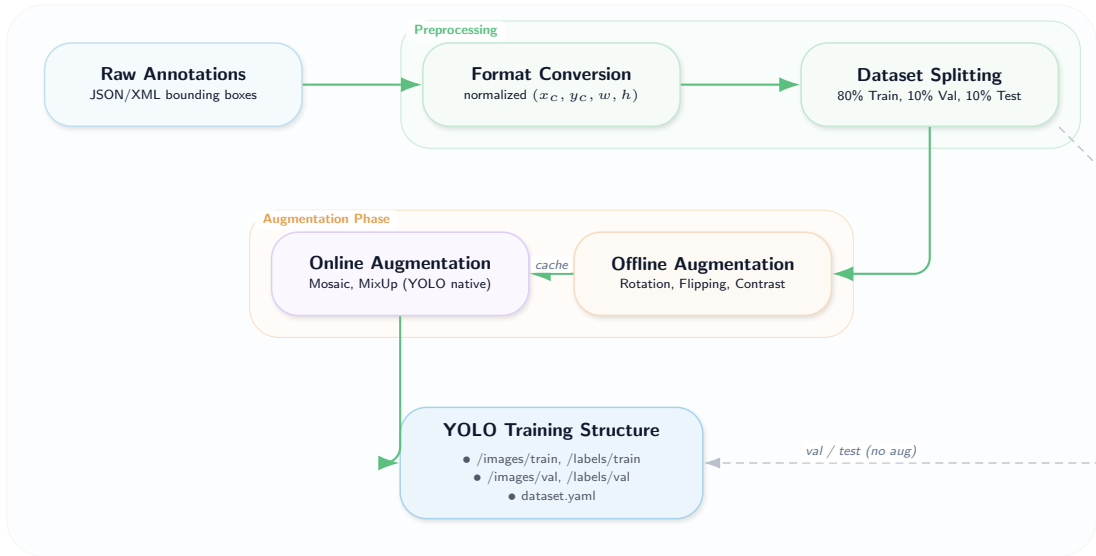
persist scan and advisory

stored record

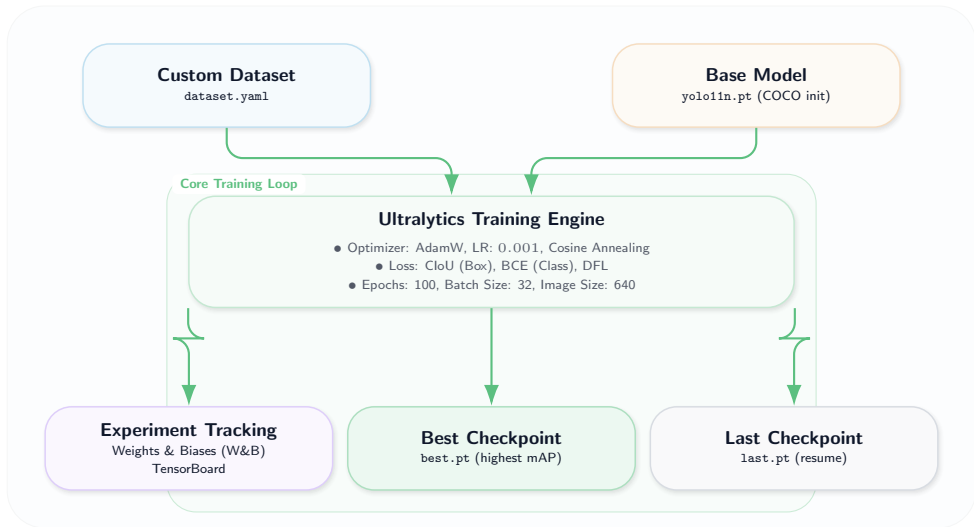
structured diagnosis response



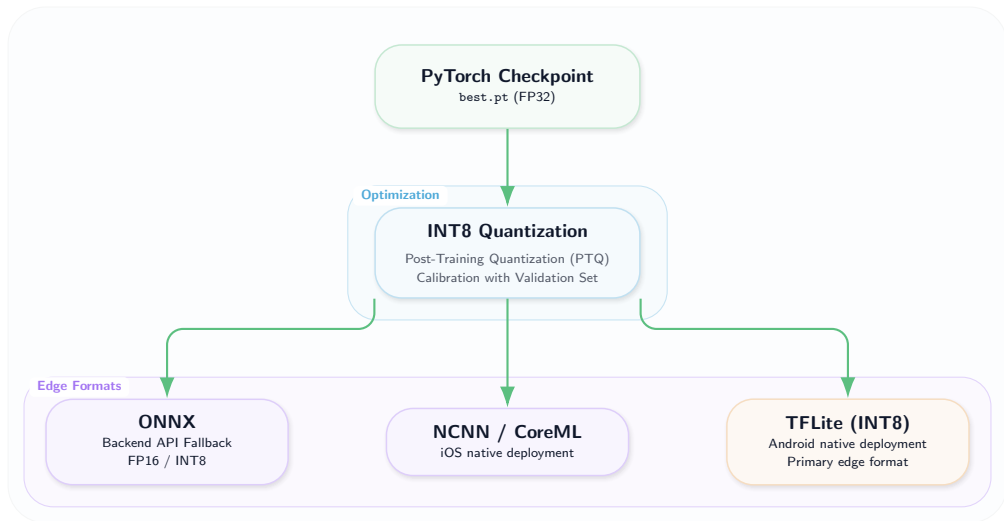




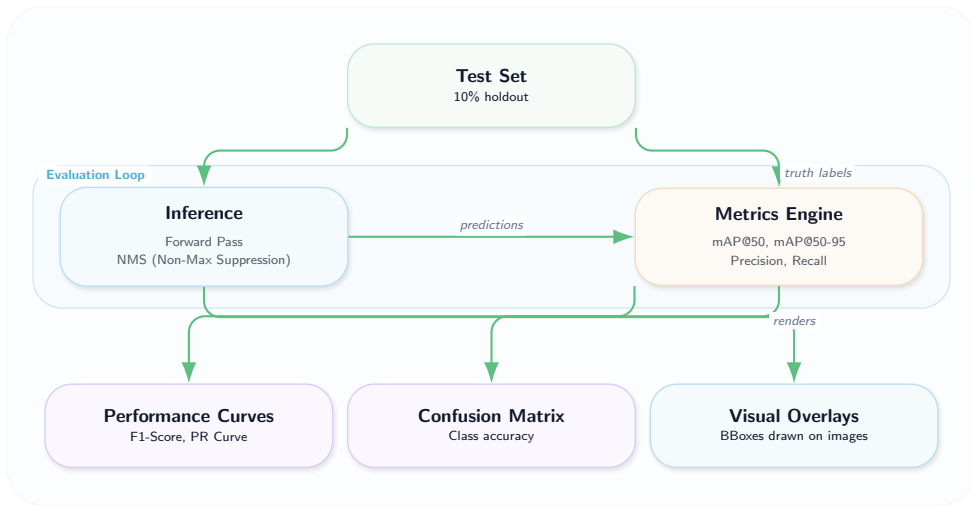
Augmentation Strategy — Offline augmentation expands the static dataset to combat class imbalance. Online augmentation (Mosaic, MixUp) is applied dynamically during YOLO training by the dataloader to improve model generalization and detection robustness on variable scales.



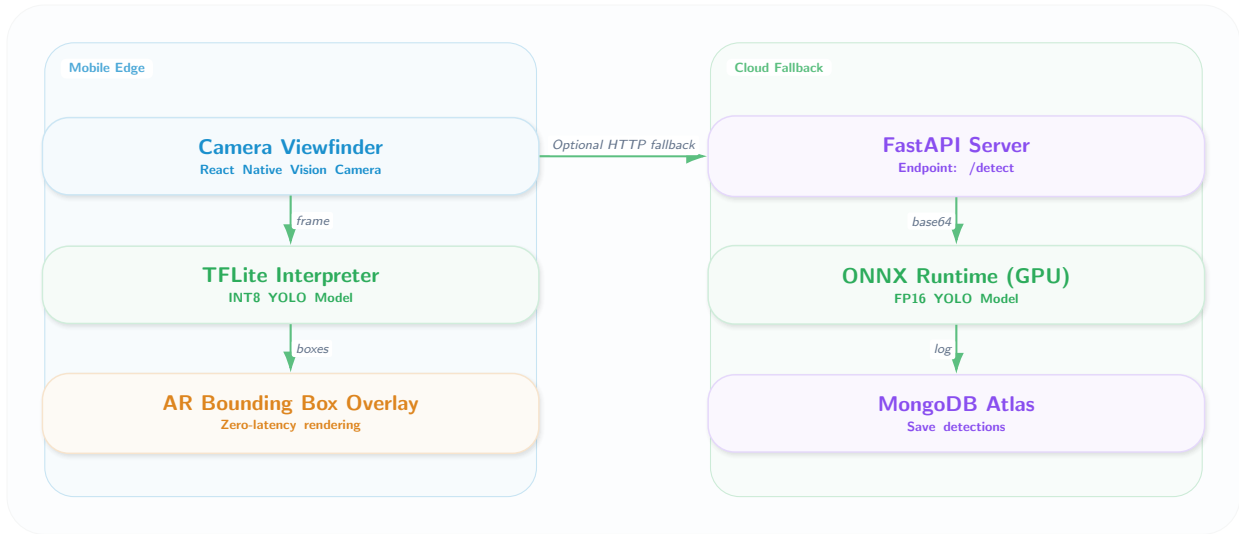
Training Dynamics — Transfer learning from COCO pre-trained weights significantly accelerates convergence. The engine tracks mAP@0.5:0.95 on the validation set at every epoch, automatically saving best.pt to prevent overfitting.



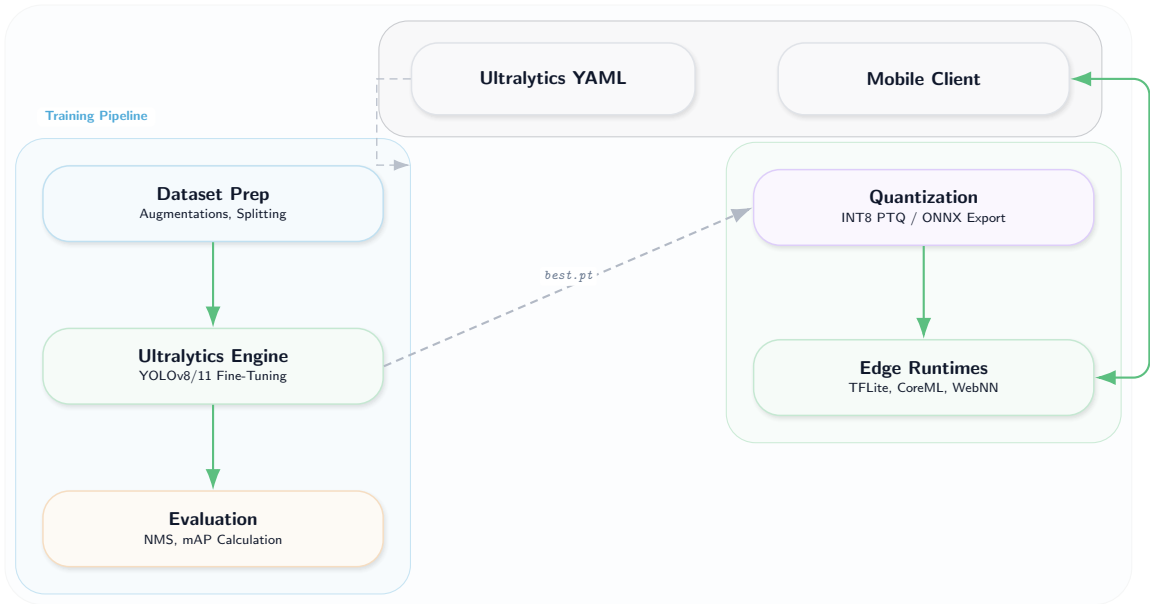
Edge Optimization — Quantizing from 32-bit floats to 8-bit integers drastically reduces model size (usually by 4x) and speeds up inference on mobile CPUs/NPUs. The validation set is used during calibration to compute the scaling factors, minimizing the inevitable drop in accuracy.



Evaluation — Non-Maximum Suppression (NMS) filters overlapping bounding boxes before predictions are compared to ground truth. Key metrics like mAP (Mean Average Precision) at various IoU thresholds provide a holistic view of the model's localization and classification accuracy.



Strategy — The primary detection runs locally on the mobile device (Edge) via TFLite, providing instant visual feedback without network latency. The backend ONNX Runtime serves as an optional high-accuracy fallback or for logging diagnostic imagery when device constraints prevent local processing.



Architecture — The YOLO architecture is cleanly split. The ML lifecycle handles the heavy lifting of data preparation and model training using PyTorch. The resultant artifacts are quantized and exported, decoupling the Python-based training from the lightweight C++/Java/Swift runtimes used on mobile edges.

Text Processing

1. Raw Ingestion

JSONL / CSV sources

2. Deduplication

URL / Hash collision

3. Text Cleaning

regex, markdown

4. Token Truncation

enforce max length

drop invalid text

Vision Validation

5. Image Download

fetch URLs, timeout

6. Format Verify

JPEG/PNG headers

7. Visual Curation

blur, resolution

drop corrupt/blur

8. Resize & Pad

conform to 1024px

Structuring & Export

9. Structuring

map to conversation

10. Tokenization

Qwen2.5 chat template

11. Packing

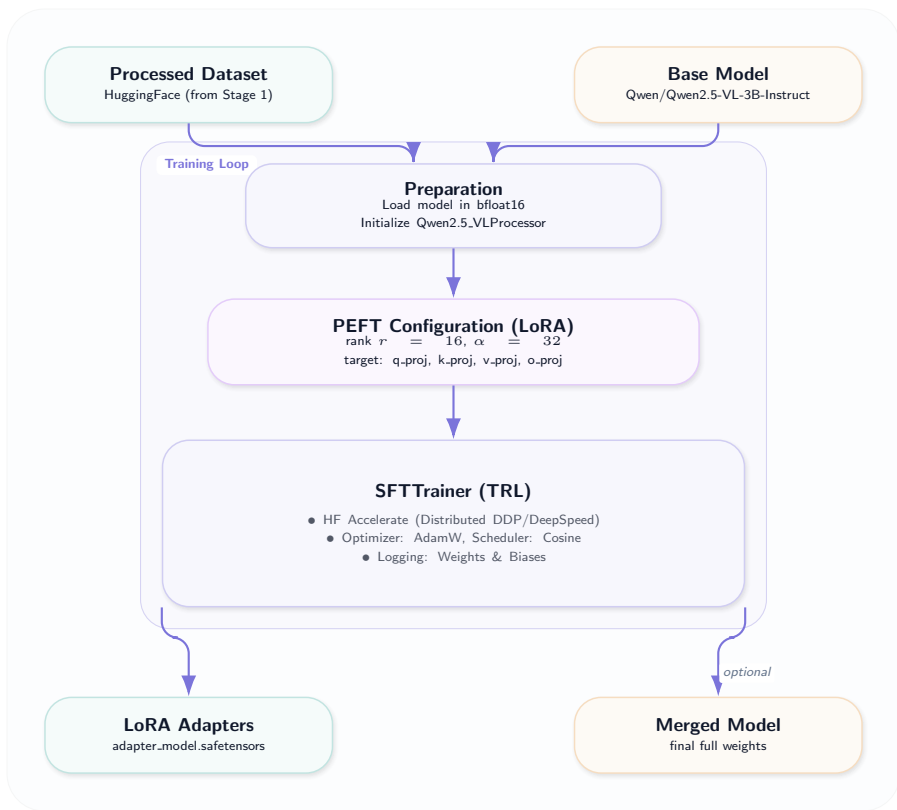
batch chunks

dataset export

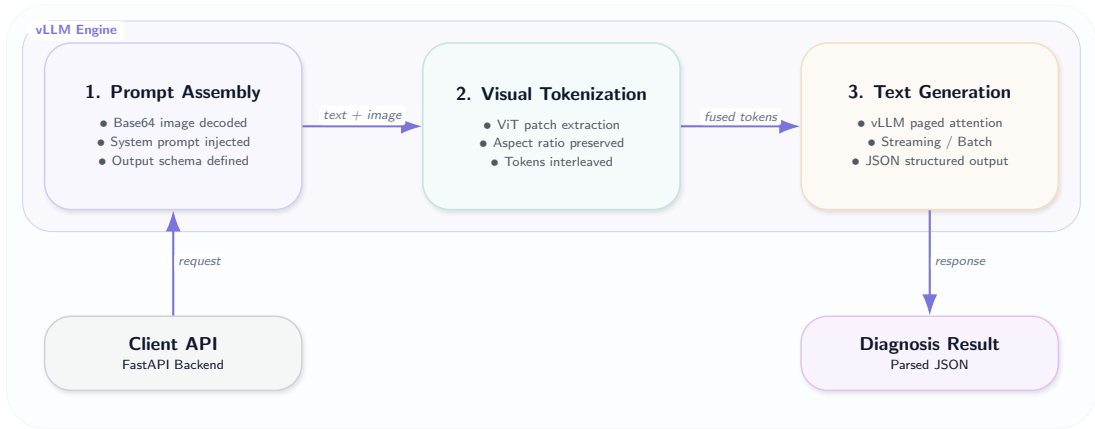
Final Dataset

HuggingFace format

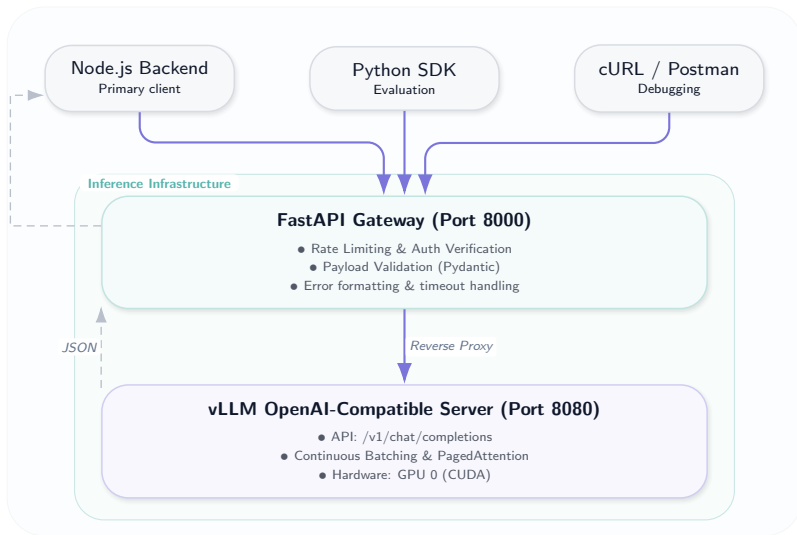
Design rationale — The pipeline is strictly sequential. Text cleaning occurs before image downloading to save bandwidth by immediately dropping malformed textual records. Visual curation guarantees that the model receives only crisp, adequately sized imagery, which is critical for the visual token representations in the Qwen architecture.



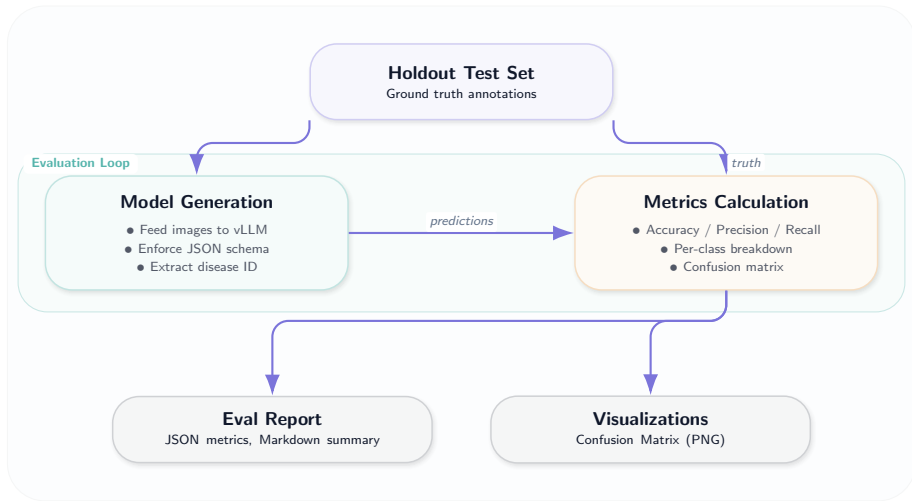
Architecture — Training leverages parameter-efficient fine-tuning (PEFT) with LoRA to adapt the vision and language projection layers without modifying the massive base model. Bfloat16 precision prevents underflow issues common in visual token scaling.



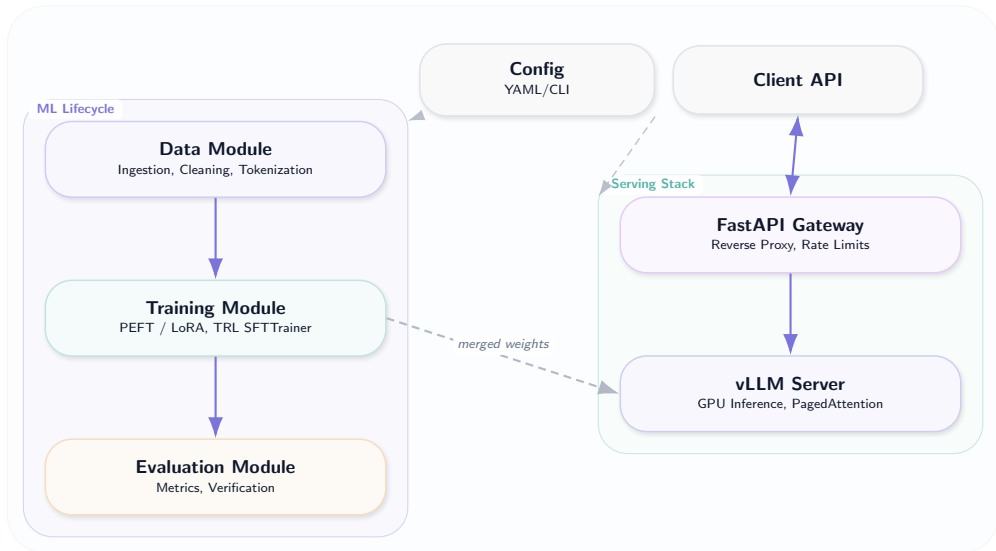
Execution — The vLLM engine hosts the merged model. Instead of stretching the image, Qwen's visual architecture dynamically tokens the image patches based on resolution, feeding them alongside the text prompt. The system instructions enforce a strict JSON structure for the backend to ingest.



Separation — The FastAPI gateway acts as a thorough front door. The internal vLLM server is never exposed directly to clients; this prevents malformed requests from crashing the GPU worker and provides a layer for metrics and API key validation.



Process — Validation ensures the LoRA weights actually learned the agricultural domain by testing against a pristine holdout set. Because generation is constrained to JSON, predictions can be programmatically compared to ground truth without human grading.



Architecture — The codebase uses a strict separation of concerns. Offline ML pipelines (data, train, eval) run independently from the online serving stack (gateway, vLLM). Configuration is injected globally via YAML.

Server Bootstrap

server.ts

1. Load .env via dotenv
2. Validate config (Zod schema)
3. Connect MongoDB Atlas
4. Register DI container (repos, services, controllers)
5. Create Express app
6. Listen on PORT

Middleware Pipeline

helmet

cors

rateLimit

body parsers

morgan

Route Dispatch

Route Matcher

/api/v1/*

no auth

authMiddleware

Public Routes

GET /health → liveness
POST /auth/* → register,
login, refresh, OTP
POST /diagnose → diagnosis

Protected Routes

JWT

/users → profile CRUD
/crops → crop management
/scans → scan records
/tasks, /notes, /advisories

Controller Layer

— validate request body → invoke service method → shape JSON response

Processing Chain

Service

business rules

DTO

Repository

data access

ODM

MongoDB

document store

catch

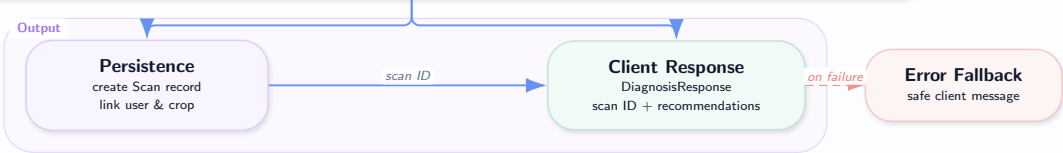
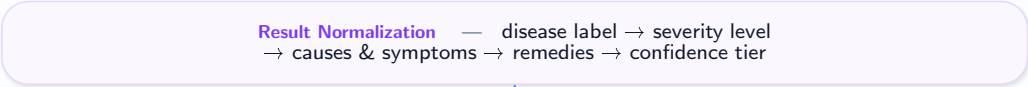
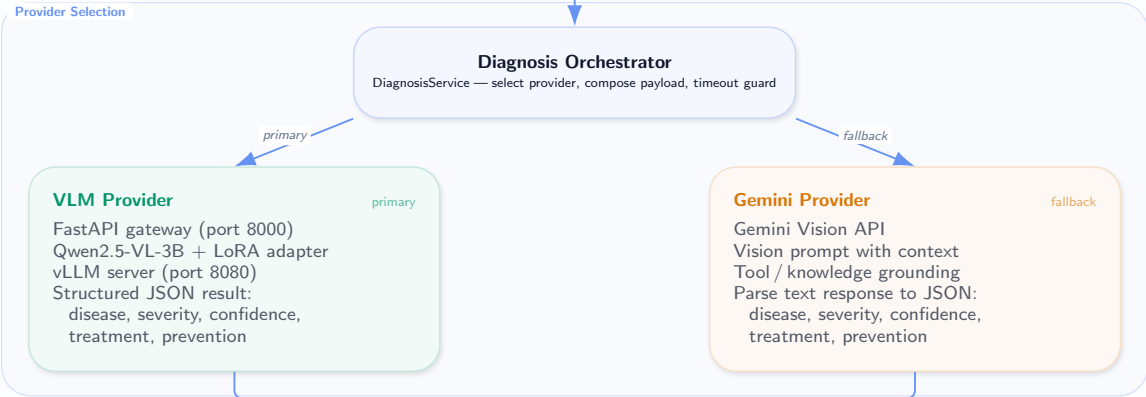
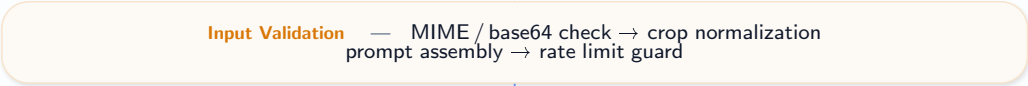
Error Handler

ApiError → JSON

JSON Response

structured envelope

Design rationale — Middleware executes in strict registration order. Public routes (/health, /diagnose, /auth/*) bypass JWT verification; protected routes inject req.user via authMiddleware. Controller → Service → Repository → MongoDB enforces separation of validation, business logic, and persistence.



Critical path — The backend hides provider differences behind a single `DiagnosisService` interface so the mobile client receives one stable diagnosis contract regardless of which inference engine produced the result. VLM is the primary provider; Gemini is activated as a fallback when the VLM gateway is unreachable or returns an error.

MongoDB Schemas

User
profile, hash

Crop
plot, variety

Scan
image, result

Task
field work

Note
observation

Advisory
guidance

OTP
recovery

RefreshToken
session

Repository Layer — `BaseRepository<T>` provides typed CRUD helpers; concrete repos (`UserRepo`, `CropRepo`, `ScanRepo`, `TaskRepo`, `NoteRepo`, `AdvisoryRepo`) add domain-specific queries

DI Container

register repos → services → controllers
single construction point at bootstrap

Service Layer

AuthService
tokens, OTP

UserService
profile ops

CropService
plot CRUD

ScanService
diagnosis

Data Paths

Write Path

validate DTO → service rule
→ repo mutation → model save

Read Path

route params → repo query
→ ownership check → serialize

MongoDB Atlas

MONGO_URI — connection lifecycle, indexes, production datastore

Architecture — Controllers stay thin; services own business rules; repositories isolate persistence mechanics behind `BaseRepository<T>`. The DI container wires everything at bootstrap so each layer depends only on abstractions, not concrete implementations. JWT user context from `req.user` propagates through the service layer for ownership checks.

Identity Establishment

Registration

Validate identity fields
Hash password (bcrypt)
Create User document
Optional OTP email

credentials

Login

Verify email + password
Issue access token (JWT)
Store refresh token
Return user profile

JWT issued

Protected Request

Extract Bearer token
Decode & verify JWT
Attach req.user
Proceed to controller

token ops

forgot password

middleware result

Token & Recovery Lifecycle

Token Lifecycle

Short-lived access token
Refresh token rotation
Logout single session
Logout all sessions
Invalidate on password change

token reset

Password Recovery

Send OTP to email
Verify OTP code
Reset password
Invalidate stale tokens
Rate-limited attempts

re-auth

Auth Outcome

200 authorized controller call
401 missing/expired token
403 invalid ownership
429 rate limit exceeded
Global error handler

Middleware Decision

authMiddleware

Decode JWT → verify expiry → lookup user → attach req.user → next()

valid

Authorized
controller proceeds

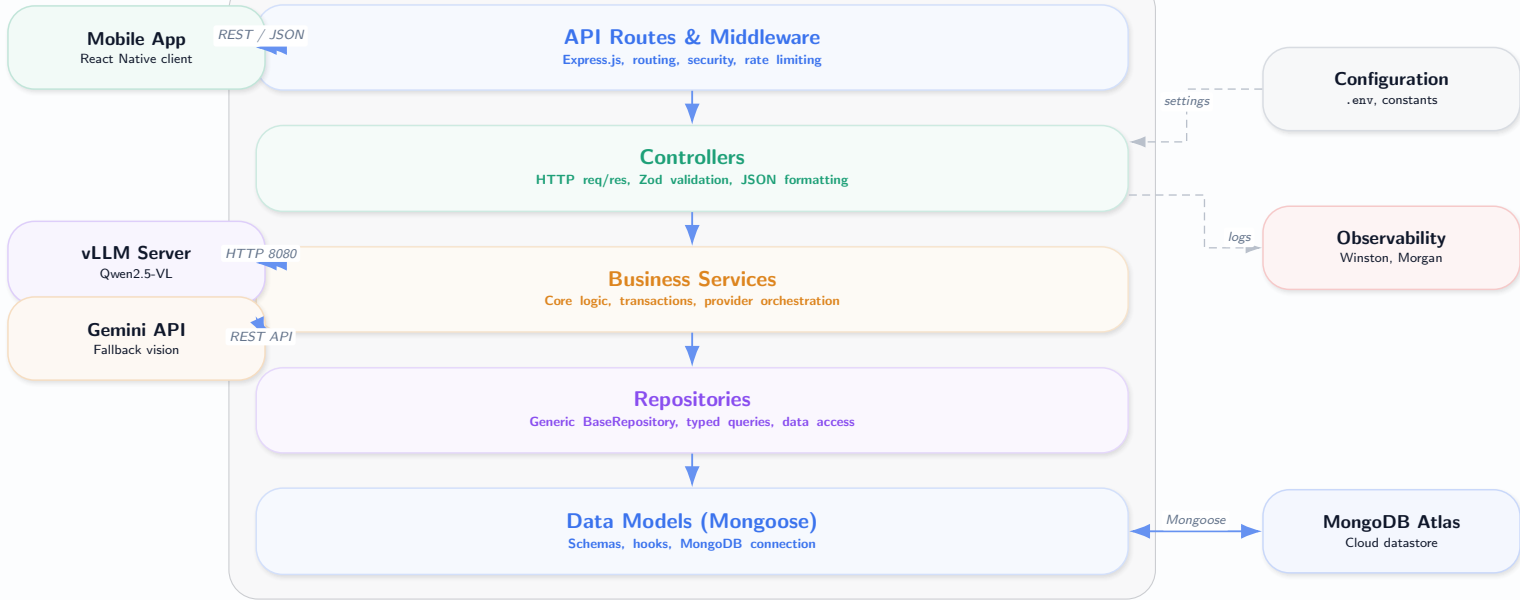
invalid

Denied

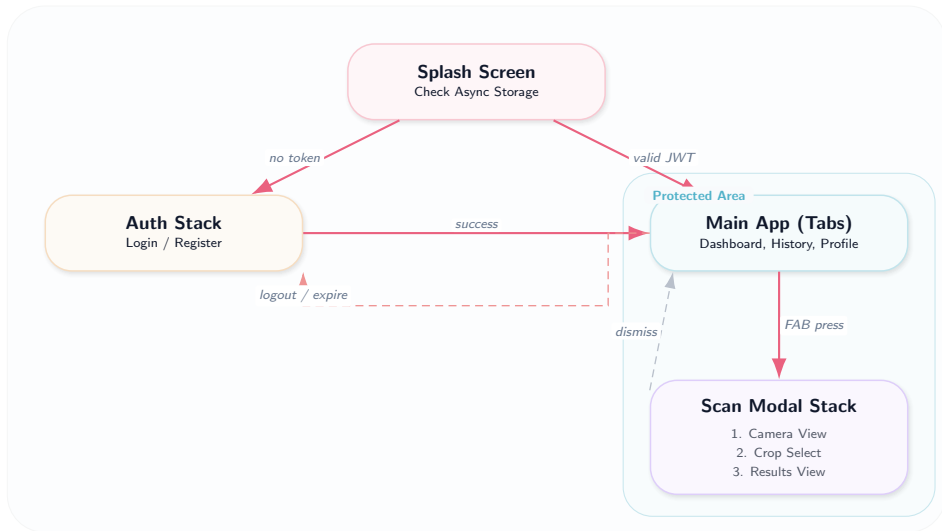
401 / 403 JSON error

Security — Session state is explicit: refresh tokens can be individually revoked without invalidating the stateless access-token verification path. The OTP recovery flow is rate-limited and time-bounded to prevent brute-force attacks. All sensitive operations log an audit trail.

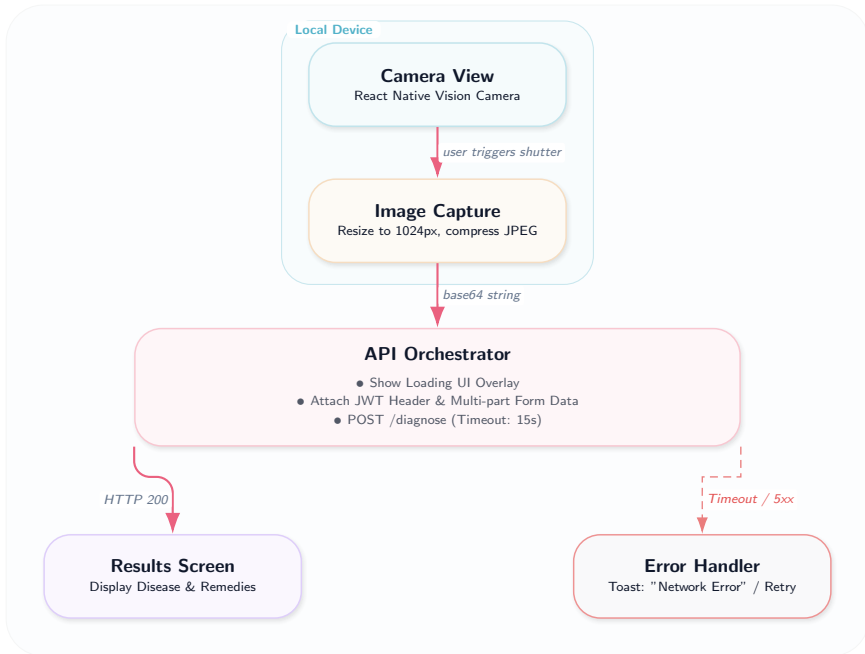
5-Layer Internal Architecture



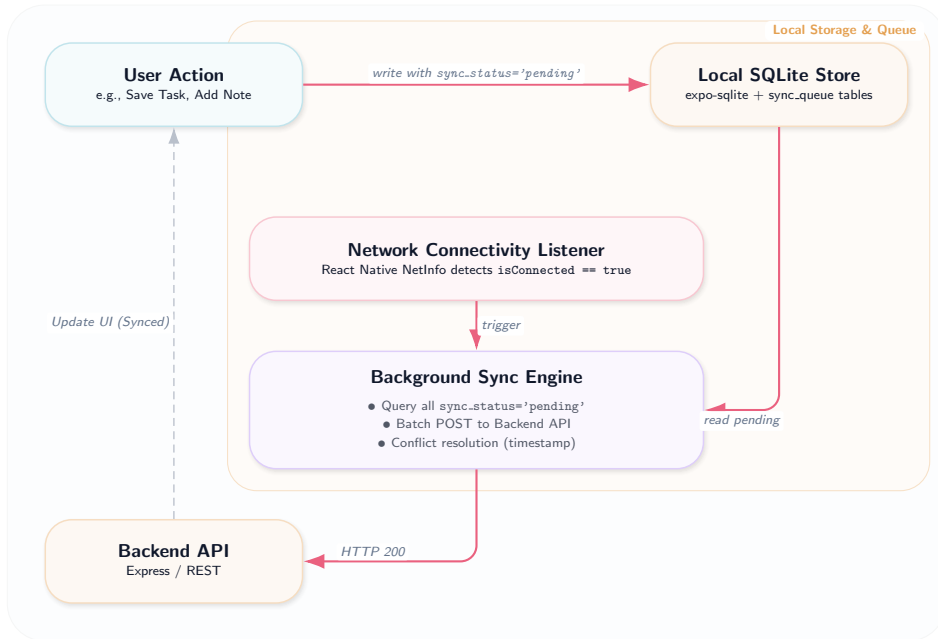
Separation of Concerns — The 5-layer architecture ensures HTTP logic (Controllers) is isolated from business logic (Services), which is isolated from database mechanics (Repositories/Models). Cross-cutting concerns like configuration and logging are injected throughout.



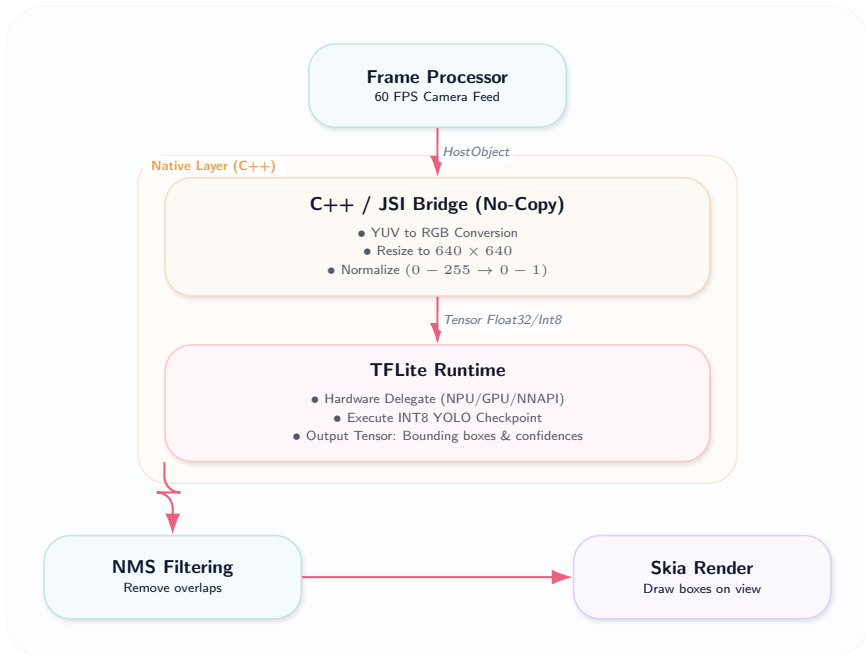
Navigation Hierarchy — The Root Navigator uses conditional rendering to swap between the Auth Stack and the Main App Stack based on the JWT state, ensuring unauthenticated users cannot deep-link into protected tabs. The Scan Stack is presented modally over the bottom tabs to preserve context.



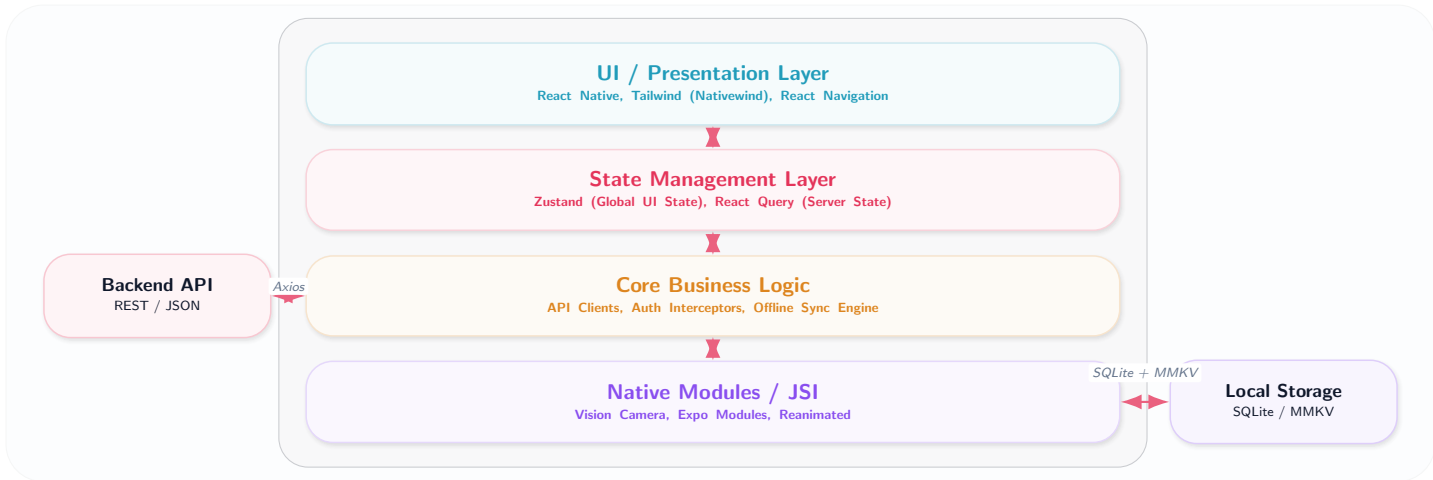
UX Considerations — Image compression happens natively on the thread to prevent UI freezing. During the API call, a custom skeleton loader or Lottie animation is shown to keep the user engaged while waiting for the VLM backend inference.



Offline-First — The UI always reads from and writes to the local SQLite-backed data layer, ensuring zero latency and full offline support. The Sync Engine runs as a background job that pushes local mutations to the cloud and pulls fresh data down when the network is available.



Performance — To achieve real-time (30+ FPS) AR overlays, frames are never serialized to base64 or passed over the React Native bridge. Instead, C++ JSI bindings manage the memory pointers directly from the camera buffer to the TFLite runtime, preventing garbage collection stutters.



Architecture — The mobile application strictly separates presentation from state and side effects. React Query handles API caching and loading states, while Zustand manages synchronous UI state. Performance-critical operations (storage, camera, ML) bypass the JS bridge using JSI.